

小清新字符串基础

NOIP String and Useless Algorithms

CaijiMK

2023 年 12 月 30 日

东威斯加里敦大学搬砖学院

前言

最小表示法

Border 理论

Z 函数与 Manacher 算法

 Z 函数

 Manacher

简单自动机入门

DFA

子序列自动机

KMP 自动机

AC 自动机

其他

Good Luck & Have Fun

前言

- 本次讲课主要内容是一些简单基础的字符串算法。

- 本次讲课主要内容是一些简单基础的字符串算法。
- 其中 Border 理论为本次讲课的重点。

- 本次讲课主要内容是一些简单基础的字符串算法。
- 其中 Border 理论为本次讲课的重点。
- 东西比较杂。

- 本次讲课主要内容是一些简单基础的字符串算法。
- 其中 Border 理论为本次讲课的重点。
- 东西比较杂。
- 例题都比较偏经典入门（板）。

- 本次讲课主要内容是一些简单基础的字符串算法。
- 其中 Border 理论为本次讲课的重点。
- 东西比较杂。
- 例题都比较偏经典入门（板）。
- 讲课人很菜，你忍一下。

- **字符集**为一个建立了全序关系的集合，用符号 Σ 表示。其中的元素被称为**字符**。如无特殊说明，则默认 $\Sigma = \{a, b, \dots, z\}$ 。

- **字符集**为一个建立了全序关系的集合，用符号 Σ 表示。其中的元素被称为**字符**。如无特殊说明，则默认 $\Sigma = \{a, b, \dots, z\}$ 。
- **字符串** s 是一个由若干字符按一定顺序排列而成的序列。

- **字符集**为一个建立了全序关系的集合，用符号 Σ 表示。其中的元素被称为**字符**。如无特殊说明，则默认 $\Sigma = \{a, b, \dots, z\}$ 。
- **字符串** s 是一个由若干字符按一定顺序排列而成的序列。
- 本课件中默认字符串的索引从 1 开始。

- **字符集**为一个建立了全序关系的集合，用符号 Σ 表示。其中的元素被称为**字符**。如无特殊说明，则默认 $\Sigma = \{a, b, \dots, z\}$ 。
- **字符串** s 是一个由若干字符按一定顺序排列而成的序列。
- 本课件中默认字符串的索引从 1 开始。
- 用 s_i 表示字符串 s 的第 i 个字符。

- **字符集**为一个建立了全序关系的集合，用符号 Σ 表示。其中的元素被称为**字符**。如无特殊说明，则默认 $\Sigma = \{a, b, \dots, z\}$ 。
- **字符串** s 是一个由若干字符按一定顺序排列而成的序列。
- 本课件中默认字符串的索引从 1 开始。
- 用 s_i 表示字符串 s 的第 i 个字符。
- 用 $|s|$ 表示字符串 s 的长度。

- **字符集**为一个建立了全序关系的集合，用符号 Σ 表示。其中的元素被称为**字符**。如无特殊说明，则默认 $\Sigma = \{a, b, \dots, z\}$ 。
- **字符串** s 是一个由若干字符按一定顺序排列而成的序列。
- 本课件中默认字符串的索引从 1 开始。
- 用 s_i 表示字符串 s 的第 i 个字符。
- 用 $|s|$ 表示字符串 s 的长度。
- **空串**即为不包含任何字符， $|s| = 0$ 的字符串。

- 定义字符与字符、字符与字符串、字符串与字符、字符串与字符串之间的**加法**表示将这两个字符或字符串按顺序拼接起来得到的字符串。

- 定义字符与字符、字符与字符串、字符串与字符、字符串与字符串之间的**加法**表示将这两个字符或字符串按顺序拼接起来得到的字符串。
- 字符串 s 的一个**子串** $s_{[l,r]}$ 表示 $s_l, s_{l+1}, \dots, s_r$ 按顺序连接得到的字符串, 即 $s_l + s_{l+1} + \dots + s_r$ 。有时也用 $s_{[l,r]} (l > r)$ 来表示空串。

- 定义字符与字符、字符与字符串、字符串与字符、字符串与字符串之间的**加法**表示将这两个字符或字符串按顺序拼接起来得到的字符串。
- 字符串 s 的一个**子串** $s_{[l,r]}$ 表示 $s_l, s_{l+1}, \dots, s_r$ 按顺序连接得到的字符串, 即 $s_l + s_{l+1} + \dots + s_r$ 。有时也用 $s_{[l,r]} (l > r)$ 来表示空串。
- 字符串 s 的一个**子序列**是由 s 中的某些字符按原先的顺序拼接得到的字符串。

- 字符串 s 的后缀为从某个位置 i 开始到 s 的最后一个字符的子串，用 $\text{suf}(s, i)$ 表示，即 $\text{suf}(s, i) = s_{i..|s|}$ 。真后缀为不是其本身的后缀，空串为任意字符串的后缀。

- 字符串 s 的后缀为从某个位置 i 开始到 s 的最后一个字符的子串，用 $\text{suf}(s, i)$ 表示，即 $\text{suf}(s, i) = s_{i, |s|}$ 。真后缀为不是其本身的后缀，空串为任意字符串的后缀。
- 字符串 s 的前缀为从 s 第一个字符开始到某个位置 i 的子串，用 $\text{pre}(s, i)$ 表示，即 $\text{pre}(s, i) = s_{1, i}$ 。真前缀为不是其本身的前缀，空串为任意字符串的前缀。

- 字符串 s 的后缀为从某个位置 i 开始到 s 的最后一个字符的子串，用 $\text{suf}(s, i)$ 表示，即 $\text{suf}(s, i) = s_{i, |s|}$ 。真后缀为不是其本身的后缀，空串为任意字符串的后缀。
- 字符串 s 的前缀为从 s 第一个字符开始到某个位置 i 的子串，用 $\text{pre}(s, i)$ 表示，即 $\text{pre}(s, i) = s_{1, i}$ 。真前缀为不是其本身的前缀，空串为任意字符串的前缀。
- **字典序**：以第 i 个字符作为第 i 关键字进行大小比较。若字符串 s_1 为字符串 s_2 的前缀，则认为 $s_1 < s_2$ 。

- 字符串 s 的后缀为从某个位置 i 开始到 s 的最后一个字符的子串，用 $\text{suf}(s, i)$ 表示，即 $\text{suf}(s, i) = s_{i, |s|}$ 。真后缀为不是其本身的后缀，空串为任意字符串的后缀。
- 字符串 s 的前缀为从 s 第一个字符开始到某个位置 i 的子串，用 $\text{pre}(s, i)$ 表示，即 $\text{pre}(s, i) = s_{1, i}$ 。真前缀为不是其本身的前缀，空串为任意字符串的前缀。
- **字典序**：以第 i 个字符作为第 i 关键字进行大小比较。若字符串 s_1 为字符串 s_2 的前缀，则认为 $s_1 < s_2$ 。
- **回文串 (Palindrome String)** 为满足将字符串倒序排列后与原串相同的串。形式化地，回文串即为满足 $\forall 1 \leq i \leq |s|, s_i = s_{|s|-i+1}$ 的字符串。

最小表示法

- **循环同构**: 若存在一个 $1 < i \leq |s|$, 使得 $\text{suf}(s, i) + \text{pre}(s, i - 1) = t$, 则称 s 与 t 循环同构。

- **循环同构**: 若存在一个 $1 < i \leq |s|$, 使得 $\text{suf}(s, i) + \text{pre}(s, i - 1) = t$, 则称 s 与 t 循环同构。
- **最小表示**: 一个字符串 s 的最小表示是所有与 s 同构的字符串中字典序最小的串。

最小表示法

- 首先有一个朴素的想法，我们维护两个指针 i, j ，比较以 i 为起点的串和以 j 为起点的串，然后每次将较大的串的指针往后移。

最小表示法

- 首先有一个朴素的想法，我们维护两个指针 i, j ，比较以 i 为起点的串和以 j 为起点的串，然后每次将较大的串的指针往后移。
- 该方法在随机数据下表现良好，但只要构造形如 `aaa...aab` 的串就可以将复杂度卡满。

最小表示法

- 首先有一个朴素的想法，我们维护两个指针 i, j ，比较以 i 为起点的串和以 j 为起点的串，然后每次将较大的串的指针往后移。
- 该方法在随机数据下表现良好，但只要构造形如 `aaa...aab` 的串就可以将复杂度卡满。
- 考虑如何优化。

最小表示法

- 首先有一个朴素的想法，我们维护两个指针 i, j ，比较以 i 为起点的串和以 j 为起点的串，然后每次将较大的串的指针往后移。
- 该方法在随机数据下表现良好，但只要构造形如 $aaa...aab$ 的串就可以将复杂度卡满。
- 考虑如何优化。
- 我们发现若存在 k 使得 $s_{[i, i+k-1]} = s_{[j, j+k-1]}$ ，且 $s_{i+k} > s_{j+k}$ ，那么 $\forall 0 \leq l \leq k$ ，以 $i+l$ 为起点的串都大于以 $j+l$ 为起点的串，所以可以直接将 i 移动到 $i+k+1$ 。

最小表示法

- 首先有一个朴素的想法，我们维护两个指针 i, j ，比较以 i 为起点的串和以 j 为起点的串，然后每次将较大的串的指针往后移。
- 该方法在随机数据下表现良好，但只要构造形如 `aaa...aab` 的串就可以将复杂度卡满。
- 考虑如何优化。
- 我们发现若存在 k 使得 $s_{[i, i+k-1]} = s_{[j, j+k-1]}$ ，且 $s_{i+k} > s_{j+k}$ ，那么 $\forall 0 \leq l \leq k$ ，以 $i+l$ 为起点的串都大于以 $j+l$ 为起点的串，所以可以直接将 i 移动到 $i+k+1$ 。
- 容易发现，每次 k 变成 0 时 i 或 j 会向后移动 k 。均摊后总复杂度为 $O(n)$ 。

■ 参考代码如下：

```
1  int i = 1, j = 2, k = 0;
2  while (i <= n && j <= n && k < n)
3      if (s[(i + k - 1) % n + 1] == s[(j + k - 1) % n + 1]) k++;
4      else {
5          if (s[(i + k - 1) % n + 1] > s[(j + k - 1) % n + 1]) i = (i + k) % n +
              1;
6          else j = (j + k) % n + 1;
7          if (i == j) i++;
8          k = 0;
9      }
10 int ans = min(i, j);
```

Border 理论

- 若一个正整数 $i < |s|$ 满足 $\text{pre}(s, i) = \text{suf}(s, |s| - i + 1)$, 则称 $\text{pre}(s, i)$ 为 s 的一个 Border。

- 若一个正整数 $i < |s|$ 满足 $\text{pre}(s, i) = \text{suf}(s, |s| - i + 1)$, 则称 $\text{pre}(s, i)$ 为 s 的一个 Border。
- 下文用 $\mathcal{B}(s)$ 表示 s 的所有 Border 构成的集合。

- 若一个正整数 $i < |s|$ 满足 $\text{pre}(s, i) = \text{suf}(s, |s| - i + 1)$, 则称 $\text{pre}(s, i)$ 为 s 的一个 Border。
- 下文用 $\mathcal{B}(s)$ 表示 s 的所有 Border 构成的集合。
- 若一个正整数 $k < |s|$ 满足 $\forall i \in (k, |s|], s_i = s_{i-k}$, 则称 k 为 s 的一个周期 (Period)。

- 若一个正整数 $i < |s|$ 满足 $\text{pre}(s, i) = \text{suf}(s, |s| - i + 1)$, 则称 $\text{pre}(s, i)$ 为 s 的一个 Border。
- 下文用 $\mathcal{B}(s)$ 表示 s 的所有 Border 构成的集合。
- 若一个正整数 $k < |s|$ 满足 $\forall i \in (k, |s|], s_i = s_{i-k}$, 则称 k 为 s 的一个周期 (Period)。
- 下文用 $\mathcal{P}(s)$ 表示 s 的所有 Period 构成的集合。

结论 1

若用 $\text{longest}(\mathcal{B}(s))$ 表示 $\mathcal{B}(s)$ 中最长的一个, 则有:

$$\mathcal{B}(s) = \mathcal{B}(\text{longest}(\mathcal{B}(s))) \cup \{\text{longest}(\mathcal{B}(s))\}$$

进一步可以得到, $\forall t \in \mathcal{B}(s), \mathcal{B}(t) \in \mathcal{B}(s)$ 。

- 考虑通过某种方法求出一个字符串 s 的所有前缀的最长的 Border，根据上面的性质即可得到其所有前缀的 Border 集合。

- 考虑通过某种方法求出一个字符串 s 的所有前缀的最长的 Border，根据上面的性质即可得到其所有前缀的 Border 集合。
- 我们设 π_i 表示 $\text{pre}(s, i)$ 最长的 Border 的长度，若该前缀没有 Border，则 $\pi_i = 0$ 。

- 考虑通过某种方法求出一个字符串 s 的所有前缀的最长的 Border，根据上面的性质即可得到其所有前缀的 Border 集合。
- 我们设 π_i 表示 $\text{pre}(s, i)$ 最长的 Border 的长度，若该前缀没有 Border，则 $\pi_i = 0$ 。
- 考虑如果我们已经求出了 $\pi_1 \dots \pi_{i-1}$ ，如何通过这些信息去求 π_i 。

- 考虑通过某种方法求出一个字符串 s 的所有前缀的最长的 Border，根据上面的性质即可得到其所有前缀的 Border 集合。
- 我们设 π_i 表示 $\text{pre}(s, i)$ 最长的 Border 的长度，若该前缀没有 Border，则 $\pi_i = 0$ 。
- 考虑如果我们已经求出了 $\pi_1 \dots \pi_{i-1}$ ，如何通过这些信息去求 π_i 。
- 容易发现，若 $\pi_i > 1$ ，则 $\text{pre}(s, \pi_i - 1) \in \mathcal{B}(\text{pre}(s, i - 1))$ 。

- 考虑通过某种方法求出一个字符串 s 的所有前缀的最长的 Border，根据上面的性质即可得到其所有前缀的 Border 集合。
- 我们设 π_i 表示 $\text{pre}(s, i)$ 最长的 Border 的长度，若该前缀没有 Border，则 $\pi_i = 0$ 。
- 考虑如果我们已经求出了 $\pi_1 \dots \pi_{i-1}$ ，如何通过这些信息去求 π_i 。
- 容易发现，若 $\pi_i > 1$ ，则 $\text{pre}(s, \pi_i - 1) \in \mathcal{B}(\text{pre}(s, i - 1))$ 。
- 于是我们有了一个看起来很暴力的做法：按长度从大到小遍历 $\mathcal{B}(\text{pre}(s, i - 1))$ ，记其长度为 j ，若 $s_{j+1} = s_i$ ，则 $\pi_i = j + 1$ 。若找不到满足条件的串，则 $\pi_i = [s_1 = s_i]$ 。

- 考虑通过某种方法求出一个字符串 s 的所有前缀的最长的 Border，根据上面的性质即可得到其所有前缀的 Border 集合。
- 我们设 π_i 表示 $\text{pre}(s, i)$ 最长的 Border 的长度，若该前缀没有 Border，则 $\pi_i = 0$ 。
- 考虑如果我们已经求出了 $\pi_1 \dots \pi_{i-1}$ ，如何通过这些信息去求 π_i 。
- 容易发现，若 $\pi_i > 1$ ，则 $\text{pre}(s, \pi_i - 1) \in \mathcal{B}(\text{pre}(s, i - 1))$ 。
- 于是我们有了看起来很暴力的做法：按长度从大到小遍历 $\mathcal{B}(\text{pre}(s, i - 1))$ ，记其长度为 j ，若 $s_{j+1} = s_i$ ，则 $\pi_i = j + 1$ 。若找不到满足条件的串，则 $\pi_i = [s_1 = s_i]$ 。
- 具体地，可以使用一个变量 j ，令其初值为 π_j ，每次令 $j \leftarrow \pi_j$ ，然后判断是否满足 $s_{j+1} = s_i$ ，直到满足条件或 $j = 0$ 。

- 考虑 j 每次至多增加 1，所以这个做法是线性的。

- 考虑 j 每次至多增加 1，所以这个做法是线性的。
- 参考代码如下：

```
1 // n is |s|.
2 for (int i = 2, j = 0; i <= n; i++) {
3     while (j && s[j + 1] != s[i]) j = pi[j];
4     if (s[j + 1] == s[i]) pi[i] = ++j
5 }
```

单模式字符串匹配

- 设文本串为 s , 模式串为 t , 那么可以直接在 $t + \# + s$ 上跑 KMP ($\#$ 是一个不在 s 和 t 的字符集内的字符)。匹配当且仅当 $\pi_i = |t|$ 。

单模式字符串匹配

- 设文本串为 s ，模式串为 t ，那么可以直接在 $t + \# + s$ 上跑 KMP ($\#$ 是一个不在 s 和 t 的字符集内的字符)。匹配当且仅当 $\pi_i = |t|$ 。
- 实际上并不需要拼起来，我们维护只要在 t 上跑 KMP，然后用一个指针遍历 s ，用另一个指针维护当前的 π 即可。

单模式串字符串匹配

- 设文本串为 s ，模式串为 t ，那么可以直接在 $t + \# + s$ 上跑 KMP ($\#$ 是一个不在 s 和 t 的字符集内的字符)。匹配当且仅当 $\pi_i = |t|$ 。
- 实际上并不需要拼起来，我们维护只要在 t 上跑 KMP，然后用一个指针遍历 s ，用另一个指针维护当前的 π 即可。
- 参考代码：

```
1 // n is |s|, m is |t|.
2 for (int i = 1, j = 0; i <= n; i++) {
3     while (j && t[j + 1] != s[i]) j = pi[j];
4     if (t[j + 1] == s[i])
5         if (++j == m) {
6             // ...
7             j = next[j];
8         }
9 }
```

- 如果我们将一个字符串 s 的每个前缀当作一个点，并对于每个 i ，建一条 $i \rightarrow \pi_i$ 的边，可以得到一棵根为 0 的内向树，这就是失配树。

- 如果我们将一个字符串 s 的每个前缀当作一个点，并对于每个 i ，建一条 $i \rightarrow \pi_i$ 的边，可以得到一棵根为 0 的内向树，这就是失配树。
- 容易发现一个长度为 i 的前缀的所有 Border 就是点 i 到根路径上的点对应的前缀。

弱周期引理 (Weak Periodicity Lemma)

$$p, q \in \mathcal{P}(s), p + q \leq |s| \implies \gcd(p, q) \in \mathcal{P}(s)$$

弱周期引理 (Weak Periodicity Lemma)

$$p, q \in \mathcal{P}(s), p + q \leq |s| \implies \gcd(p, q) \in \mathcal{P}(s)$$

证明

令 $p < q$, 设 $d = q - p$, 只需证 $d \in \mathcal{P}(s)$, 就可以用辗转相减法证明结论。

- $\forall i \in (d, |s| - p], s_i = s_{i+p} = s_{i+p-q} = s_{i-d}$
- $\forall i \in (q, |s|], s_i = s_{i-q} = s_{i-q+p} = s_{i-d}$

因为 $p + q \leq |s|$, 即 $q \leq |s| - p$, 所以

$(d, |s| - p] \cup (q, |s|] = (d, |s|]$ 。故 $d \in \mathcal{P}(s)$ 。命题得证。

- 一般情况下弱周期引理就够用了，但实际上有更紧的界。

- 一般情况下弱周期引理就够用了，但实际上有更紧的界。

周期引理 (Periodicity Lemma)

$$p, q \in \mathcal{P}(s), p + q - \gcd(p, q) \leq |s| \implies \gcd(p, q) \in \mathcal{P}(s)$$

周期引理

- 一般情况下弱周期引理就够用了，但实际上有更紧的界。

周期引理 (Periodicity Lemma)

$$p, q \in \mathcal{P}(s), p + q - \gcd(p, q) \leq |s| \implies \gcd(p, q) \in \mathcal{P}(s)$$

证明

令 $p < q$ ，设 $d = q - p$ 。套用弱周期引理的证明方法容易证明

$d \in \mathcal{P}(s_{[1, |s| - p]})$, $d \in \mathcal{P}(s_{(p, |s|]})$ 。由于

$p + (q - p) - \gcd(p, q - p) \leq |s| - p$ ，所以可以归纳证明
 $\gcd(p, q) \in \mathcal{P}(s_{[1, |s| - p]})$, $\gcd(p, q) \in \mathcal{P}(s_{(p, |s|]})$ 。

因为 $2p = p + q - (q - p) \leq p + q - \gcd(p, q) \leq |s|$ ，即
 $p \leq |s| - p$ ，所以有 $\gcd(p, q) \in \mathcal{P}(s_{[1, p]})$ 。

并且由于 $\gcd(p, q) | p$ ，所以 $\gcd(p, q)$ 是 $s_{[1, p]}$ 的整周期，而
 $\gcd(p, q)$ 又是 $s_{(p, |s|]}$ 的周期，所以 $\gcd(p, q)$ 是 s 的周期。

结论 2

一个字符串的 Border 与 Period 是一一对应的。具体来说，
 $\text{pre}(s, i) \in \mathcal{B}(s) \iff |s| - i \in \mathcal{P}(s)$ 。

结论 2

一个字符串的 Border 与 Period 是一一对应的。具体来说,
 $\text{pre}(s, i) \in \mathcal{B}(s) \iff |s| - i \in \mathcal{P}(s)$ 。

结论 3

若 t 是 s 的前缀, 且 $a \in \mathcal{P}(s), b \in \mathcal{P}(t), b|a$, 且 b 是 t 的整周期, 且 $|t| \geq a$, 则 $b \in \mathcal{P}(s)$ 。

结论 2

一个字符串的 Border 与 Period 是一一对应的。具体来说，
 $\text{pre}(s, i) \in \mathcal{B}(s) \iff |s| - i \in \mathcal{P}(s)$ 。

结论 3

若 t 是 s 的前缀，且 $a \in \mathcal{P}(s)$, $b \in \mathcal{P}(t)$, $b|a$ ，且 b 是 t 的整周期，
且 $|t| \geq a$ ，则 $b \in \mathcal{P}(s)$ 。

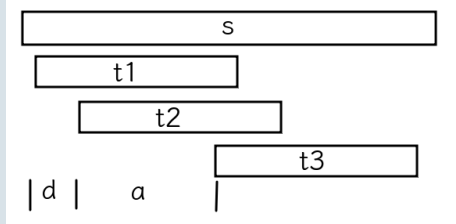
- 这两个结论证明比较简单，这里不再赘述。

结论 4

对于文本串 s 与模式串 t , 若 $|t| > \frac{|s|}{2}$, 且 t 在 s 中至少成功匹配了 3 次, 则每次匹配的位置形成一个等差数列, 公差为 t 最小的周期。

证明

考虑匹配成功的第 1 次、第 2 次和最后一次，分别记为 t_1, t_2, t_x 。



设 t_1, t_2 的间隔为 d , t_2, t_x 的间隔为 a 。

因为 $|t| > \frac{|s|}{2}$, 所以 t_1 和 t_x 的间隔不超过 $|t|$ 。

所以 $d + a \leq |t|$, 根据弱周期引理, $\gcd(d, a)$ 也是 t 的周期。

证明 · 续

设 t 最小的周期为 p ，显然 $p \leq \gcd(d, a) \leq d$ 。根据弱周期引理显然还有周期 $\gcd(p, d, a) \leq p$ 。因为 p 最小，故 $p \mid \gcd(d, a) \mid d$ 。若 $p \neq d$ ，则在与 t_1 间隔 p 的位置也会有一个匹配。故 $(p = d) \mid q$ 。

以上分析在 t_x 取其他出现位置时一样成立，故相邻两次间隔都是 p 的倍数，并且除最后一次外每次匹配成功后间隔 p 位置一定有一次成功的匹配。故所有匹配的间隔相同，即匹配位置形成等差数列。

结论 5

一个字符串 s 所有长度不小于 $\frac{|s|}{2}$ 的 Border 的长度构成一个等差数列。

结论 5

一个字符串 s 所有长度不小于 $\frac{|s|}{2}$ 的 Border 的长度构成一个等差数列。

证明

设最大的 Border 的长度为 $|s| - p$ ，另一个 Border 的长度为 $|s| - q$ ，且有 $p, q \leq \frac{|s|}{2}$ 。

由弱周期引理可知 $\gcd(p, q)$ 也是 s 的周期。故 $p \leq \gcd(p, q)$ ，即 $p|q$ 。

容易发现每次将最长的 Border 的长度减去 p 都会得到一个新的 Border，所以所有长度大于 $\frac{|s|}{2}$ 的 Border 的长度构成了一个公差为 p 的等差数列。

结论 6

一个字符串 s 的所有 Border 的长度从小到大排序后可以划分成 $O(\log |s|)$ 个连续段，且每个连续段都是一个等差数列。

结论 6

一个字符串 s 的所有 Border 的长度从小到大排序后可以划分成 $O(\log |s|)$ 个连续段, 且每个连续段都是一个等差数列。

证明

设最长的 Border 长度为 $|s| - p$, 长度不小于 $\frac{|s|}{2}$ 最小的 Border 的长度为 a 。

若 $p \leq \frac{|s|}{4}$, 则必然存在一个正整数 k 使得 $\frac{|s|}{2} \leq n - kp < \frac{3}{4}|s|$, 故 $a < \frac{3}{4}|s|$ 。

若 $p > \frac{|s|}{4}$, 则 $a \leq n - p < \frac{3}{4}|s|$ 。由于剩下的 Border 都是长度不小于 $\frac{|s|}{2}$ 最小的 Border 的 Border, 所以每次问题规模缩小为原来的 $\frac{3}{4}$, 划分次数为 $O(\log |s|)$ 。

结论 7

回文串的回文前/后缀即为该回文串的 Border。

结论 7

回文串的回文前/后缀即为该回文串的 Border。

结论 8

若回文串 s 有周期 p ，则可以把 $\text{pre}(s, p)$ 划分成长度为 $|s| \bmod p$ 的前缀和长度为 $p - |s| \bmod p$ 的后缀，使得它们都是回文串。

结论 7

回文串的回文前/后缀即为该回文串的 Border。

结论 8

若回文串 s 有周期 p ，则可以把 $\text{pre}(s, p)$ 划分成长度为 $|s| \bmod p$ 的前缀和长度为 $p - |s| \bmod p$ 的后缀，使得它们都是回文串。

结论 9

若 t 是回文串 s 的最长 Border，则 t 在 s 中只能匹配 2 次。

结论 7

回文串的回文前/后缀即为该回文串的 Border。

结论 8

若回文串 s 有周期 p ，则可以把 $\text{pre}(s, p)$ 划分成长度为 $|s| \bmod p$ 的前缀和长度为 $p - |s| \bmod p$ 的后缀，使得它们都是回文串。

结论 9

若 t 是回文串 s 的最长 Border，则 t 在 s 中只能匹配 2 次。

- 这三个证明也比较简单，留作习题。

- 记 $\text{Ssuf}(s)$ 为在 s 后接任意一个串后可能成为最小后缀的串的集合。形式化地, $t \in \text{Ssuf}(s) \iff$ 存在一个字符串 u (可以为空) 使得 $t + u$ 是 $s + u$ 的最小后缀。

- 记 $\text{Ssuf}(s)$ 为在 s 后接任意一个串后可能成为最小后缀的串的集合。形式化地, $t \in \text{Ssuf}(s) \iff$ 存在一个字符串 u (可以为空) 使得 $t + u$ 是 $s + u$ 的最小后缀。

结论 10

对于任意一个字符串 s 以及任意 $u, v \in \text{Ssuf}(s), |u| < |v|$, 一定有 u 是 v 的 Border。

结论 11

对于任意一个字符串 s 以及任意 $u, v \in \text{Ssuf}(s)$, $|u| < |v|$, 一定有 $2|u| < |v|$ 。

结论 11

对于任意一个字符串 s 以及任意 $u, v \in \text{Ssuf}(s)$, $|u| < |v|$, 一定有 $2|u| < |v|$ 。

证明

由结论 10 可得 u 是 v 的一个 Border。

设与 u 对应的周期为 p , 记 $t = \text{pre}(v, p)$, 则 u, v 可以表示为 $u = t + w, v = t + t + w$ 。

若存在一个字符串 r 使得 $u + r < v + r$, 则

$t + w + r < t + t + w + r$, 即 $w + r < t + w + r = u + r$, 故 $u \notin \text{Ssuf}(s)$, 矛盾。

结论 11

对于任意一个字符串 s 以及任意 $u, v \in \text{Ssuf}(s)$, $|u| < |v|$, 一定有 $2|u| < |v|$ 。

证明

由结论 10 可得 u 是 v 的一个 Border。

设与 u 对应的周期为 p , 记 $t = \text{pre}(v, p)$, 则 u, v 可以表示为 $u = t + w, v = t + t + w$ 。

若存在一个字符串 r 使得 $u + r < v + r$, 则

$t + w + r < t + t + w + r$, 即 $w + r < t + w + r = u + r$, 故 $u \notin \text{Ssuf}(s)$, 矛盾。

结论 12

$|\text{Ssuf}(s)| \leq \log_2 |s|$ 。

Z 函数与 Manacher 算法

- 对于一个字符串 s ，定义函数 $z(i)$ 为 s 与 $\text{suf}(s, i)$ 的最长公共前缀 (Longest Common Prefix, LCP)。

- 对于一个字符串 s , 定义函数 $z(i)$ 为 s 与 $\text{suf}(s, i)$ 的最长公共前缀 (Longest Common Prefix, LCP)。
- 特别地, $z(1) = 0$ 。

- 相信大家都会。

- 假设我们已经求出 $z(1), z(2), \dots, z(i-1)$, 考虑如何根据这些信息求出 $z(i)$ 。

- 假设我们已经求出 $z(1), z(2), \dots, z(i-1)$, 考虑如何根据这些信息求出 $z(i)$ 。
- 我们取 $j + z(j)$ 最大的一个位置, 若 $j + z(j)$, 若 $i < j + z(j)$, 我们可以得到 $s_{[i, j+z(j)]} = s_{[i-j+1, z(j)]}$ 。

- 假设我们已经求出 $z(1), z(2), \dots, z(i-1)$, 考虑如何根据这些信息求出 $z(i)$ 。
- 我们取 $j + z(j)$ 最大的一个位置, 若 $j + z(j)$, 若 $i < j + z(j)$, 我们可以得到 $s_{[i, j+z(j)]} = s_{[i-j+1, z(j)]}$ 。
- 因此 $z(i) \geq \min\{z(i-j+1), j + z(j) - i\}$ 。

- 假设我们已经求出 $z(1), z(2), \dots, z(i-1)$, 考虑如何根据这些信息求出 $z(i)$ 。
- 我们取 $j + z(j)$ 最大的一个位置, 若 $j + z(j)$, 若 $i < j + z(j)$, 我们可以得到 $s_{[i, j+z(j)]} = s_{[i-j+1, z(j)]}$ 。
- 因此 $z(i) \geq \min\{z(i-j+1), j + z(j) - i\}$ 。
- 考虑从前往后维护 j , 每次若 $i < j + z(j)$ 则 $z(i)$ 以 $\min\{z(i-j+1), j + z(j) - i\}$ 为初值, 之后继续暴力匹配。

- 看起来只是优化常数？

- 看起来只是优化常数？
- 实际上若 $i \geq j + z(j)$ 或 $j + z(j) - i \leq z(i - j + 1)$ 都会使 $j + z(j)$ 的值增大至少 1，而 $i < j + z(j)$ 且 $z(i - j + 1) < j + z(j) - i$ 时有 $z(i) = z(i - j + 1)$ 。

- 看起来只是优化常数？
- 实际上若 $i \geq j + z(j)$ 或 $j + z(j) - i \leq z(i - j + 1)$ 都会使 $j + z(j)$ 的值增大至少 1，而 $i < j + z(j)$ 且 $z(i - j + 1) < j + z(j) - i$ 时有 $z(i) = z(i - j + 1)$ 。
- 因此总时间复杂度为 $O(|s|)$ 。

■ 参考代码如下：

```
1 // n is |s|.
2 for (int i = 2, j = 0; i <= n; i++) {
3     if (i < j + z[j]) z[i] = min(z[i - j + 1], j + z[j] - i);
4     while (i + z[i] <= n && s[i + z[i]] == s[1 + z[i]]) z[i]++;
5     if (i + z[i] > j + z[j]) j = i;
6 }
```

- Manacher 用于求一个字符串 s 的所有回文子串。

- Manacher 用于求一个字符串 s 的所有回文子串。
- 显然等价于对于每个回文中心求其对应的最长回文子串的长度。

- Manacher 用于求一个字符串 s 的所有回文子串。
- 显然等价于对于每个回文中心求其对应的最长回文子串的长度。
- 但是长度为偶数的串有点难搞，于是我们考虑将所有回文串的长度都变成奇数。

- Manacher 用于求一个字符串 s 的所有回文子串。
- 显然等价于对于每个回文中心求其对应的最长回文子串的长度。
- 但是长度为偶数的串有点难搞，于是我们考虑将所有回文串的长度都变成奇数。
- 方法是对于串 $s = s_1 s_2 \dots s_n$ ，将其变成 $s = \# s_1 \# s_2 \# \dots \# s_n \#$ 。

- Manacher 用于求一个字符串 s 的所有回文子串。
- 显然等价于对于每个回文中心求其对应的最长回文子串的长度。
- 但是长度为偶数的串有点难搞，于是我们考虑将所有回文串的长度都变成奇数。
- 方法是对于串 $s = s_1 s_2 \dots s_n$ ，将其变成
 $s = \# s_1 \# s_2 \# \dots \# s_n \#$ 。
- 这样所有长度为偶数的串的中心就变成了某个 $\#$ ，并且若现在求得的最长回文子串长度为 p ，则原本长度就是 $\lfloor \frac{p}{2} \rfloor$ ，也等于新串的「半径」。

- 相信大家都会。

- 考虑用类似的套路优化，设 p_j 为以 j 为回文中心的最长回文子串的半径，假设已经求出了 $p_1, p_2, \dots, p_{i-1}$ ，取 $j + p_j$ 最大的 j 。

- 考虑用类似的套路优化，设 p_j 为以 j 为回文中心的最长回文子串的半径，假设已经求出了 $p_1, p_2, \dots, p_{i-1}$ ，取 $j + p_j$ 最大的 j 。
- 若 $i \leq j + p_j$ ，则有 $p_i \geq \min\{p_{2j-i}, j + p_j - i + 1\}$ 。

- 考虑用类似的套路优化，设 p_j 为以 j 为回文中心的最长回文子串的半径，假设已经求出了 $p_1, p_2, \dots, p_{i-1}$ ，取 $j + p_j$ 最大的 j 。
- 若 $i \leq j + p_j$ ，则有 $p_i \geq \min\{p_{2j-i}, j + p_j - i + 1\}$ 。
- 以此为 p_i 的初值，然后暴力更新。

- 和 Z 函数的复杂度分析是一样的，相信大家都会。

- 注意实现时一般在前后另添加两个不同的特殊字符防止越界。

```
1 // t is the original string, n is |t|.
2 string s = "^#";
3 for (char i : t) s.push_back(i), s.push_back('#');
4 s.push_back('@');
5 for (int i = 1, j = 0; i <= 2 * n + 1; i++) {
6     if (i <= j + p[j]) p[i] = min(p[2 * j - i], j + p[j] - i);
7     while (s[i - p[i] - 1] == s[i + p[i] + 1]) p[i]++;
8     if (i + p[i] > j + p[j]) j = i;
9 }
```

简单自动机入门

- 本次讲课只涉及有限状态自动机 (Finite Automaton, FA)。

- 本次讲课只涉及有限状态自动机 (Finite Automaton, FA)。
- 「自动机是一个对信号序列进行判定的数学模型。」——OI Wiki

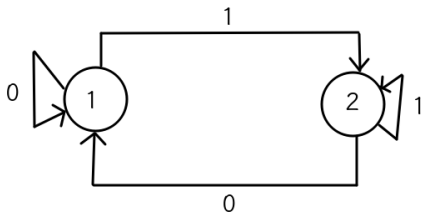
- 本次讲课只涉及**有限状态自动机 (Finite Automaton, FA)**。
- 「自动机是一个对信号序列进行判定的数学模型。」——OI Wiki
- 其中**信号序列**是指一串有顺序的信号，如一个字符串、一个数组等。

- 本次讲课只涉及**有限状态自动机 (Finite Automaton, FA)**。
- 「自动机是一个对信号序列进行判定的数学模型。」——OI Wiki
- 其中**信号序列**是指一串有顺序的信号，如一个字符串、一个数组等。
- **判定**指的是针对某个命题的真或假的回答，如判定一个数是奇数还是偶数。

- 本次讲课只涉及**有限状态自动机 (Finite Automaton, FA)**。
- 「自动机是一个对信号序列进行判定的数学模型。」——OI Wiki
- 其中**信号序列**是指一串有顺序的信号，如一个字符串、一个数组等。
- **判定**指的是针对某个命题的真或假的回答，如判定一个数是奇数还是偶数。
- 自动机的大致原理是一个有向图，从某些点出发并维护一个指针，根据信号序列上指针所指向的位置的值在图上游走并向后移动指针，然后根据最终所在的点进行判定。

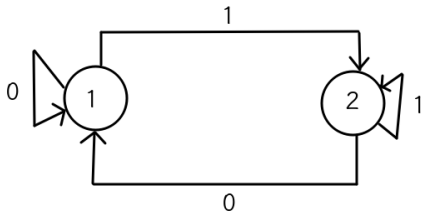
- 听起来有点抽象？不妨来看下面这个例子：

- 听起来有点抽象？不妨来看下面这个例子：



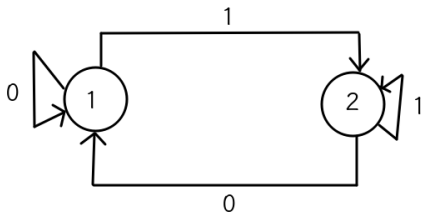
■

- 听起来有点抽象？不妨来看下面这个例子：



- 这是一个用于判定一个数奇偶的自动机，我们以数的二进制从高位到低位作为信号序列，从点 1 出发，然后根据当前位在图上游走，若最后位于 1 号节点则这个数是偶数。

- 听起来有点抽象？不妨来看下面这个例子：



- 这是一个用于判定一个数奇偶的自动机，我们以数的二进制从高位到低位作为信号序列，从点 1 出发，然后根据当前位在图上游走，若最后位于 1 号节点则这个数是偶数。
- 我们将这张有向图上的节点称为**状态**，边称为**转移**。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。
 - Σ : 字符集, 该自动机只能输入这些字符。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。
 - Σ : 字符集, 该自动机只能输入这些字符。
 - Q : 状态集合。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。
 - Σ : 字符集, 该自动机只能输入这些字符。
 - Q : 状态集合。
 - s : 该自动机的起始状态, 判定时从起始状态出发, $s \in Q$ 。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。
 - Σ : 字符集, 该自动机只能输入这些字符。
 - Q : 状态集合。
 - s : 该自动机的起始状态, 判定时从起始状态出发, $s \in Q$ 。
 - F : 该自动机的接受状态集合, 若最终停留在接受状态则认为自动机接受了这个信号序列, 也就是判定这个命题为真, $F \subseteq Q$ 。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。
 - Σ : 字符集, 该自动机只能输入这些字符。
 - Q : 状态集合。
 - s : 该自动机的起始状态, 判定时从起始状态出发, $s \in Q$ 。
 - F : 该自动机的接受状态集合, 若最终停留在接受状态则认为自动机接受了这个信号序列, 也就是判定这个命题为真, $F \subseteq Q$ 。
 - δ : 转移函数, 是一个映射 $(Q, \Sigma) \rightarrow Q$, 在转移时输入当前状态和信号序列上当前位置的值, 得到下一个状态。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。
 - Σ : 字符集, 该自动机只能输入这些字符。
 - Q : 状态集合。
 - s : 该自动机的起始状态, 判定时从起始状态出发, $s \in Q$ 。
 - F : 该自动机的接受状态集合, 若最终停留在接受状态则认为自动机接受了这个信号序列, 也就是判定这个命题为真, $F \subseteq Q$ 。
 - δ : 转移函数, 是一个映射 $(Q, \Sigma) \rightarrow Q$, 在转移时输入当前状态和信号序列上当前位置的值, 得到下一个状态。
- 若一个自动机 A 接受了字符串 s , 则 $A(s) = \text{True}$, 否则 $A(s) = \text{False}$ 。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。
 - Σ : 字符集, 该自动机只能输入这些字符。
 - Q : 状态集合。
 - s : 该自动机的起始状态, 判定时从起始状态出发, $s \in Q$ 。
 - F : 该自动机的接受状态集合, 若最终停留在接受状态则认为自动机接受了这个信号序列, 也就是判定这个命题为真, $F \subseteq Q$ 。
 - δ : 转移函数, 是一个映射 $(Q, \Sigma) \rightarrow Q$, 在转移时输入当前状态和信号序列上当前位置的值, 得到下一个状态。
- 若一个自动机 A 接受了字符串 s , 则 $A(s) = \text{True}$, 否则 $A(s) = \text{False}$ 。
- 另外, 若一个状态 u 没有字符 c 的转移, 则我们认为 $\delta(u, c) = \text{null}$ 。null 是一个特殊的状态, 它只能转移到自己且不属于接受状态。

- 确定有限状态自动机 (Deterministic Finite Automaton, DFA) 可以被表示为 $(\Sigma, Q, s, F, \delta)$ 的形式。
 - Σ : 字符集, 该自动机只能输入这些字符。
 - Q : 状态集合。
 - s : 该自动机的起始状态, 判定时从起始状态出发, $s \in Q$ 。
 - F : 该自动机的接受状态集合, 若最终停留在接受状态则认为自动机接受了这个信号序列, 也就是判定这个命题为真, $F \subseteq Q$ 。
 - δ : 转移函数, 是一个映射 $(Q, \Sigma) \rightarrow Q$, 在转移时输入当前状态和信号序列上当前位置的值, 得到下一个状态。
- 若一个自动机 A 接受了字符串 s , 则 $A(s) = \text{True}$, 否则 $A(s) = \text{False}$ 。
- 另外, 若一个状态 u 没有字符 c 的转移, 则我们认为 $\delta(u, c) = \text{null}$ 。null 是一个特殊的状态, 它只能转移到自己且不属于接受状态。
- 下面会介绍一些在 OI 中常见的 DFA。

- 子序列自动机是一个只能接受某个字符串的子序列的自动机。

子序列自动机

- 子序列自动机是一个只能接受某个字符串的子序列的自动机。
- 对于一个串 s ，其有 $|s| + 1$ 个状态。不妨设起始状态为 0，状态 i 表示 $s_{[1,i]}$ 与 $s_{[1,i-1]}$ 的子序列的差集。

- 子序列自动机是一个只能接受某个字符串的子序列的自动机。
- 对于一个串 s ，其有 $|s| + 1$ 个状态。不妨设起始状态为 0，状态 i 表示 $s_{[1,i]}$ 与 $s_{[1,i-1]}$ 的子序列的差集。
- 于是我们可以定义子序列自动机：

$$Q = \{x \in \mathbb{N} \mid 0 \leq x \leq |s|\}$$

$$s = 0$$

$$F = Q$$

$$\delta(u, c) = \begin{cases} \min\{i \mid i > u, s_i = c\} & \exists i > u, s_i = c \\ \text{null} & \text{otherwise} \end{cases}$$

- 只要从后往前枚举，维护每个字符最后一次出现（即靠前一次出现）即可。

- 只要从后往前枚举，维护每个字符最后一次出现（即靠前一次出现）即可。
- 时间复杂度 $O(|s||\Sigma|)$ 。

- KMP 是一个接受所有以模式串 s 为后缀的字符串的自动机。

- KMP 是一个接受所有以模式串 s 为后缀的字符串的自动机。
- 具体来说就是考虑 KMP 匹配的过程，然后建成自动机。

- KMP 是一个接受所有以模式串 s 为后缀的字符串的自动机。
- 具体来说就是考虑 KMP 匹配的过程，然后建成自动机。
- 于是我们就可以得到：

$$Q = \{x \in \mathbb{N} \mid 0 \leq x \leq |s|\}$$

$$s = 0$$

$$F = |s|$$

$$\delta(u, c) = \begin{cases} u + 1 & u \neq |s| \wedge s_{u+1} = c \\ \delta(\pi(u), c) & u = |s| \vee u \neq 0 \wedge s_{u+1} \neq c \\ 0 & u = 0 \wedge s_{u+1} \neq c \end{cases}$$

- KMP 是一个接受所有以模式串 s 为后缀的字符串的自动机。
- 具体来说就是考虑 KMP 匹配的过程，然后建成自动机。
- 于是我们就可以得到：

$$Q = \{x \in \mathbb{N} \mid 0 \leq x \leq |s|\}$$

$$s = 0$$

$$F = |s|$$

$$\delta(u, c) = \begin{cases} u + 1 & u \neq |s| \wedge s_{u+1} = c \\ \delta(\pi(u), c) & u = |s| \vee u \neq 0 \wedge s_{u+1} \neq c \\ 0 & u = 0 \wedge s_{u+1} \neq c \end{cases}$$

- 显然可以通过 KMP 算法在 $O(|s||\Sigma|)$ 的时间复杂度内构建。

- 相较于 KMP 算法，KMP 自动机每次转移都是 $O(1)$ 的，复杂度不基于均摊，但是我并没有找到什么 KMP 自动机能做 KMP 做不了的题。

- 相较于 KMP 算法，KMP 自动机每次转移都是 $O(1)$ 的，复杂度不基于均摊，但是我并没有找到什么 KMP 自动机能做 KMP 做不了的题。
- 于是我们得出结论：~~所以 KMP 自动机在 $O(1)$ 中没有应用。~~

- 一组模式串的 AC 自动机是一个接受所有以某个模式串为后缀的串的自动机。

AC 自动机

- 一组模式串的 AC 自动机是一个接受所有以某个模式串为后缀的串的自动机。
- 看起来是 KMP 自动机的强化版，考虑用类似的思路去构建。

AC 自动机

- 一组模式串的 AC 自动机是一个接受所有以某个模式串为后缀的串的自动机。
- 看起来是 KMP 自动机的强化版，考虑用类似的思路去构建。
- 首先对于模式串建出 Trie，则 Q 为 Trie 上所有节点， s 为 Trie 的根（记为 0）， F 为所有表示某个模式串节点。

- 一组模式串的 AC 自动机是一个接受所有以某个模式串为后缀的串的自动机。
- 看起来是 KMP 自动机的强化版，考虑用类似的思路去构建。
- 首先对于模式串建出 Trie，则 Q 为 Trie 上所有节点， s 为 Trie 的根（记为 0）， F 为所有表示某个模式串的点。
- 类似于 KMP 的 π 函数，我们定义 $fail_i$ 表示 Trie 上 i 表示的串的最长真后缀对应的串（称其为 fail 指针）。

- 一组模式串的 AC 自动机是一个接受所有以某个模式串为后缀的串的自动机。
- 看起来是 KMP 自动机的强化版，考虑用类似的思路去构建。
- 首先对于模式串建出 Trie，则 Q 为 Trie 上所有节点， s 为 Trie 的根（记为 0）， F 为所有表示某个模式串的节点。
- 类似于 KMP 的 π 函数，我们定义 $fail_i$ 表示 Trie 上 i 表示的串的最长真后缀对应的串（称其为 fail 指针）。
- 若我们可以求出 fail 指针，则可以得到其转移函数：

$$\delta(u, c) = \begin{cases} trie_{u,c} & trie_{u,c} \neq \text{null} \\ \delta(fail_u, c) & u \neq 0 \wedge trie_{u,c} = \text{null} \\ 0 & u = 0 \wedge trie_{u,c} = \text{null} \end{cases}$$

其中 $trie_{u,c}$ 表示 Trie 树上节点 u 通过字符 c 走到的儿子，若不存在则为 null。

- 发现 $\text{fail}_{\text{trie}_{u,c}} = \delta(\text{fail}_u, c)$ 。

- 发现 $\text{fail}_{\text{trie}_{u,c}} = \delta(\text{fail}_u, c)$ 。
- 于是我们考虑同时求 fail 指针和转移函数，这就要求我们规划一个合理的求解顺序。

- 发现 $\text{fail}_{\text{trie}_{u,c}} = \delta(\text{fail}_u, c)$ 。
- 于是我们考虑同时求 fail 指针和转移函数，这就要求我们规划一个合理的求解顺序。
- 容易发现 fail_u 的深度一定小于 u 的深度，所以考虑从 Trie 的根开始 bfs 求解，然后就做完了。

- 发现 $\text{fail}_{\text{trie}_{u,c}} = \delta(\text{fail}_u, c)$ 。
- 于是我们考虑同时求 fail 指针和转移函数，这就要求我们规划一个合理的求解顺序。
- 容易发现 fail_u 的深度一定小于 u 的深度，所以考虑从 Trie 的根开始 bfs 求解，然后就做完了。
- 时间复杂度为 $O(|s||\Sigma|)$ 。

- 对于每个点 $u \neq 0$, 连一条 u 到 fail_u , 可以得到一棵以 0 为根的内向。

- 对于每个点 $u \neq 0$ ，连一条 u 到 fail_u ，可以得到一棵以 0 为根的内向。
- fail 树上每个节点的父亲都是该串的真后缀，基于这一点，很多题目都可以通过将询问离线下来转化成树上问题的方式优化复杂度。

- 还是来考虑那个经典的问题吧，给你一个文本串和若干模式串，求出每个模式串在文本串中的出现次数。

- 还是来考虑那个经典的问题吧，给你一个文本串和若干模式串，求出每个模式串在文本串中的出现次数。
- 首先对模式串建出 AC 自动机，然后用文本串在自动机上跑。

- 还是来考虑那个经典的问题吧，给你一个文本串和若干模式串，求出每个模式串在文本串中的出现次数。
- 首先对模式串建出 AC 自动机，然后用文本串在自动机上跑。
- 任意时刻，当前节点及其所有在 fail 树上的祖先对应的串都是当前扫到的文本串的后缀。

- 还是来考虑那个经典的问题吧，给你一个文本串和若干模式串，求出每个模式串在文本串中的出现次数。
- 首先对模式串建出 AC 自动机，然后用文本串在自动机上跑。
- 任意时刻，当前节点及其所有在 fail 树上的祖先对应的串都是当前扫到的文本串的后缀。
- 也就是说每次走到一个节点 u ，所有 u 的祖先的答案都会增加 1。

- 还是来考虑那个经典的问题吧，给你一个文本串和若干模式串，求出每个模式串在文本串中的出现次数。
- 首先对模式串建出 AC 自动机，然后用文本串在自动机上跑。
- 任意时刻，当前节点及其所有在 fail 树上的祖先对应的串都是当前扫到的文本串的后缀。
- 也就是说每次走到一个节点 u ，所有 u 的祖先的答案都会增加 1。
- 直接对每个点单点修改最后求子树和即可。

其他

- WC2016 论战捆竹竿
- 洛谷 P9348 小园香径独徘徊
- NOI2011 阿狸的打字机

- Periodicity Lemma - 知乎
- Border 理论小记 - command_block 的博客 - 洛谷博客
- 自动机 - OI Wiki
- Aho-Corasick algorithm - Algorithms for Competitive Programming
- 各种祖传课件……

Good Luck & Have Fun
